

Desafio Final: Criptografia



Visão Geral do Desafio

Este é o projeto final da disciplina de Segurança da Informação. Em **duplas**, vocês desenvolverão um aplicativo de mensagens em rede que demonstre na prática os conceitos de criptografia estudados no Capítulo 3.

O sistema deve permitir que dois usuários se comuniquem de forma segura, utilizando criptografia híbrida (RSA + AES), autenticação com hash de senhas, e comunicação via sockets em Python.



Conceitos Aplicados: Criptografia Simétrica (AES), Criptografia Assimétrica (RSA), Funções Hash (SHA-256), Autenticação, Sockets TCP/IP



Objetivos de Aprendizagem

- ✓ Implementar troca segura de chaves usando criptografia assimétrica (RSA)
- ✓ Utilizar criptografia simétrica (AES) para criptografar as mensagens em tempo real
- ✓ Aplicar funções hash (SHA-256) para armazenamento seguro de senhas
- ✓ Desenvolver comunicação em rede usando sockets TCP em Python
- ✓ Compreender e implementar o conceito de criptografia híbrida
- ✓ Demonstrar a importância da segurança nas comunicações digitais



Funcionalidades Obrigatórias



Autenticação com Hash

Sistema de login onde as senhas são armazenadas apenas como hash SHA-256. Nunca armazene senhas em texto claro!

SHA-256

Salting



Troca de Chaves RSA

Cada usuário gera um par de chaves RSA. As chaves públicas são trocadas para estabelecer o canal seguro.

RSA-2048

Assimétrica

Criptografia AES

As mensagens são criptografadas com AES-256 em modo CBC. A chave AES é criptografada com RSA e enviada ao destinatário.

AES-256

CBC

Comunicação por Sockets

Servidor e cliente comunicam via TCP sockets. O servidor roteia mensagens entre os usuários conectados.

TCP/IP

Python Socket



Arquitetura do Sistema

O sistema deve seguir a arquitetura cliente-servidor com o seguinte fluxo de segurança:

Fluxo de Comunicação Segura





Orientações Técnicas

Estrutura de Arquivos Sugerida

secure_chat/

- └─  server.py (servidor de roteamento)
- └─  client.py (cliente com interface)
- └─  crypto_utils.py (funções de criptografia)
- └─  auth.py (autenticação com hash)
- └─  database.py (armazenamento de usuários)
- └─  requirements.txt (dependências)

Fluxo de Autenticação

1 **Cadastro do Usuário**

A senha fornecida pelo usuário é processada com SHA-256 + salt único antes de armazenar no banco de dados.

2 **Login**

O hash da senha digitada é comparado com o hash armazenado. Nunca há decodificação — apenas comparação.

3 Geração de Chaves RSA

Após autenticar, o cliente gera um par de chaves RSA (pública + privada) para a sessão.

4 Troca de Chaves

O servidor distribui as chaves públicas entre os clientes conectados.

Fluxo de Mensagens

1 Geração da Chave AES

O remetente gera uma chave AES aleatória para criptografar a mensagem.

2 Criptografia da Mensagem

A mensagem é criptografada com AES-256-CBC usando a chave gerada.

3 Criptografia da Chave AES

A chave AES é criptografada com a chave pública RSA do destinatário.

4 Envio ao Servidor

O servidor roteia a mensagem criptografada e a chave AES protegida para o destinatário.

5 Descriptografia

O destinatário decifra a chave AES com sua chave privada RSA, depois decifra a mensagem.

 **Importante:** Use a biblioteca `cryptography` do Python para implementação das cifras. Não implemente algoritmos criptográficos do zero — utilize bibliotecas auditadas e testadas.



Referências de Implementação

Autenticação com Hash SHA-256

```
import hashlib
import os

def hash_password(password):
    salt = os.urandom(16)
    pwd_hash = hashlib.pbkdf2_hmac('sha256',
                                    password.encode(), salt, 100000)
    return salt + pwd_hash
```

Geração de Chaves RSA

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

# Gerar par de chaves RSA-2048
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
public_key = private_key.public_key()
```

Criptografia AES com Chave RSA

```
from cryptography.hazmat.primitives.ciphers import Cipher,
algorithms, modes
import os

# Gerar chave AES aleatória
aes_key = os.urandom(32) # AES-256
iv = os.urandom(16)

# Criptografar mensagem com AES
cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv))
encryptor = cipher.encryptor()
ciphertext = encryptor.update(message) + encryptor.finalize()
```



Bibliografia: Consulte a documentação oficial da biblioteca `cryptography` do Python e os slides do Capítulo 3 de Criptografia.



Critérios de Avaliação

Requisito	Descrição	Pontos
Autenticação	Login com hash SHA-256 + salt, armazenamento seguro de senhas	20 pts
RSA	Geração de chaves, troca segura, criptografia da chave AES	25 pts
AES	Criptografia/descriptografia de mensagens com AES-256-CBC	25 pts

Requisito	Descrição	Pontos
Sockets	Comunicação TCP funcional, servidor roteando mensagens	20 pts
Código	Organização, documentação, tratamento de erros	10 pts
GitHub	Repositório público com código, README, histórico de commits	10 pts
Vídeo	Vídeo no YouTube (5-10 min) explicando e demonstrando o sistema	10 pts
Bônus	Interface gráfica, histórico de mensagens, múltiplos clientes	+5 pts



Entregáveis

Código Fonte (GitHub)

- ✓ Repositório público no GitHub com todos os arquivos
- ✓ Arquivo `requirements.txt` com dependências
- ✓ README.md com instruções de execução e screenshots
- ✓ Histórico de commits organizado (commits semânticos)

- ✓ .gitignore adequado (não incluir chaves ou .env)

Vídeo Demonstrativo (YouTube)

- ✓ Vídeo de 5 a 10 minutos no YouTube
- ✓ Explicação da arquitetura e fluxo do sistema
- ✓ Demonstração prática: login + envio de mensagens
- ✓ Mostrar que as mensagens estão criptografadas (嗅探 com Wireshark)
- ✓ Explicar as escolhas técnicas (RSA, AES, Hash)

Checklist GitHub

- ✓ Repositório público
- ✓ README.md completo
- ✓ Código organizado em pastas
- ✓ Pelo menos 10 commits
- ✓ .gitignore configurado

Checklist Vídeo

- ✓ Tempo: 5-10 minutos
- ✓ Apresentação da dupla
- ✓ Demonstração ao vivo
- ✓ Explicação técnica
- ✓ Link no README do GitHub



Prazo: 2 semanas a partir da data de entrega do desafio.



Entregas: Link do repositório GitHub + link do vídeo YouTube via formulário online.



Nota: A nota será composta pela avaliação do código (70%), vídeo (20%) e documentação (10%).



Dicas Importantes



Segurança

Nunca envie chaves privadas pela rede. Use OAEP para padding RSA e CBC com IV aleatório para AES.



Dependências

Use `pip install cryptography` — é a biblioteca padrão para criptografia em Python.



Depuração

Teste cada módulo separadamente antes de integrar. Comece pelo hash, depois RSA, depois AES.



Recursos

Consulte os slides do Capítulo 3 e a documentação oficial do Python para exemplos.

Segurança da Informação — IFPI

Desafio Final: Criptografia | 2025

"Criptografia é como um cofre: sem a chave certa, o conteúdo é inútil"